Non Structural Persistent Operating System Interceptor: Observation-First Computing for a Single-Surface Interface

Kundai Farai Sachikonye
AIMe Registry for Artificial Intelligence
kundai.sachikonye@wzw.tum.de

April 18, 2026

Abstract

Modern graphical user interfaces assume a multi-surface presentation model in which many elements (windows, panels, menus, toolbars) are simultaneously visible and the user navigates among them to locate the relevant affordance. This model imposes measurable cognitive and motor overhead, quantified by Fitts's and Hick's laws, on every interaction. We develop an alternative: a *single-surface interface* in which the user is presented at any time with at most one primary element — a text prompt, or a rendered artifact produced by the system in response to a prior utterance. The single-surface model is not a regression to text-only command lines; it is the native presentation model of a computing architecture we call *observation-first*. The architecture rests on a mathematical identity, derived from first principles, between observation and computation in bounded systems: every computation that produces a result is equivalent to an observation that resolves the same categorical address in a structured state space. From this identity we derive two principal consequences. First, inter-layer communication within the architecture cannot implement request/response semantics without thermodynamic overhead; instead, layers compose partial observations, and the user's visual system completes the chain. Second, file operations in such a system are symmetric: saving and retrieving are the same primitive (coordinate extraction) differentiated only by the sign of a single axis. We formalize the identity, prove the no-RPC theorem, and establish completeness of a four-state single-surface presentation machine. We argue that scientific research is the canonical application domain: scientific questions are natively observational, their answers are determined by underlying structure, and the desktop metaphor has proved a poor fit for scientific workflow. We specify empirical protocols for validating the cognitive cost comparison and discuss implications for interface design.

1 Introduction

1.1 The Desktop Metaphor and Its Costs

The graphical desktop has been the dominant paradigm of personal computing for four decades [Kay and Goldberg, 1977, Smith et al., 1982, Apple Computer, 1985]. Its central idea is a two-dimensional spatial presentation of many simultaneously-accessible elements — files, applications, dialogs — that the user navigates with a pointing device. The paradigm has been enormously successful in making computers usable by non-technical audiences [Shneiderman, 1983].

The paradigm also has costs, and those costs are quantifiable. Every interaction with a desktop interface involves at least two sub-operations: first, *locating* the target element on the two-dimensional surface, and second, *acting* on it. Locating is dominated by Fitts's law [Fitts, 1954], which quantifies motor time for pointing as $T = a + b\log_2(D/W + 1)$, where D is distance to target and W is target width. Acting, when it involves selecting among alternatives (menu items, toolbar buttons), is dominated by Hick's law [Hick, 1952], which gives decision time as $T = a + b\log_2(n + 1)$ for n

alternatives.

These laws describe lower bounds on human interaction time under the multi-surface model. They are not tuning parameters. For typical desktop tasks, Fitts's law accounts for approximately 230ms per pointing operation, Hick's law for approximately 150ms per decision among ~ 10 alternatives, with a floor of ~ 400ms per action. Multi-step tasks accumulate these costs; empirical studies of common desktop workflows show 2.7–5.7s of unavoidable motor and cognitive cost per task, with real measurements of 5–15s dominated by this floor [Card et al., 1983].

This cost is intrinsic to the multi-surface presentation model, not to the pointing device or graphical rendering. No refinement of the desktop — better icons, more accessible menus, gesture shortcuts — can beat the Fitts–Hick floor without changing the presentation model.

1.2 An Observation-First Alternative

We propose a different presentation model: the *single-surface interface*, in which at most one primary element is visible at any time. The user types a natural-language utterance into a prompt; the system interprets the utterance and either renders an artifact or returns to blank if no artifact is warranted. The user is not asked to navigate a two-dimensional space of simultaneously-visible objects; they describe what they want, and the system shows what (if anything) is to be shown.

This is superficially similar to a command-line interface, but it differs in three important ways:

1. The user is not restricted to a fixed command grammar; utterances are free-form natural language.

2. The system does not return textual command output; it renders whatever the utterance calls for, including graphical content.

3. The interface has explicit presentation state beyond "text in, text out": the single surface cycles through blank, prompt, artifact, and artifact-plus-prompt states according to a well-defined policy.

We call this the *blank-screen paradigm*. We argue that it is not a regression but the native presentation model for a class of computational architectures we call *observation-first*.

1.3 The Observation-Computation Identity

Observation-first computing rests on a mathematical identity: in any bounded system admitting a structured state space, the operations of *observing* a system state, *computing* that state from prior conditions, and *processing* data to extract the state are mathematically identical. All three reduce to resolving a categorical address in a partition of the state space.

We derive this identity from first principles in Sections 2–3, using only standard statistical mechanics [Boltzmann, 1872, Shannon, 1948] and Koopman operator theory [Koopman, 1931, Mezić, 2005]. The identity is not novel in spirit — philosophers of computation have long argued something similar [Chaitin, 1975, Wheeler, 1990] — but our derivation specifies the structural conditions under which it holds and the algorithmic consequences that follow.

1.4 Main Consequences

From the observation-computation identity, two consequences flow directly:

The no-RPC theorem: Inter-layer communication in an observation-first stack cannot implement request/response semantics with zero additional cost. Layers must compose partial observations, where each layer refines a single observation in progress rather than serving requests from higher layers.

File-operation symmetry: Operations on persistent data (saving and retrieving) are the same primitive — coordinate extraction — differentiated only by the sign of a single axis. There is no filesystem exposed to the user; storage is described semantically, and retrieval uses the same semantic description.

Together with the single-surface presentation model, these consequences define a computational architecture substantially different from conventional systems, whose implications for user experience (particularly for scientific research) we develop in Section 9.

1.5 Contributions

1. A first-principles derivation of the observation-computation identity for bounded systems (Section 3).

2. A formal model of partial observations and their composition (Section 4).

3. The *no-RPC theorem*: characterization of inter-layer semantics in observation-first architectures (Section 5).

4. The four-state single-surface presentation machine with completeness and soundness results (Section 6).

5. The *rendering identity*: a formal statement that rendering pixels for presentation is computationally identical to observing a partition state (Section 7).

6. The *file-operation symmetry theorem*: save and retrieve reduce to the same coordinate-extraction primitive (Section 8).

7. An analysis of scientific research workflow as the canonical observation-first workload (Section 9).

8. Cognitive-cost comparison against GUI and CLI for representative scientific tasks (Section 10).

9. Empirical validation protocols (Section 11).

1.6 Organization

Section 2 develops the necessary structural background on bounded systems. Section 3 derives the observation-computation identity. Section 4 formalizes partial observations. Section 5 proves the no-RPC theorem. Section 6 presents the single-surface presentation machine. Section 7 establishes the rendering identity. Section 8 proves file-operation symmetry. Section 9 discusses scientific research workflow. Section 10 analyzes cognitive cost. Section 11 specifies experiments. Section 12 surveys related work. Section 13 discusses implications. Section 14 concludes.

2 Bounded Systems and Their State Spaces

2.1 Bounded Systems

Our theoretical framework concerns *bounded systems*: physical or computational systems with finite energy, finite volume, and finite time horizon. The thermodynamic boundedness of physical systems follows from the Bekenstein–Hawking entropy bound [Bekenstein, 1973, Hawking, 1975], which limits the number of distinguishable states in a region of space-time to

$$N_{\max} = \frac{2\pi RE}{\hbar c}, \tag{1}$$

where R is the system's characteristic radius and E its energy. For any physical system of macroscopic but finite extent, $N_{\max}$ is astronomically large but finite.

Computational boundedness is easier: any computing system realized with finite hardware has finitely many distinguishable configurations. We assume this throughout.

Axiom 2.1 (System Boundedness)**.** The systems we consider have finitely many distinguishable states.

2.2 State Partitions

A natural organizational principle for bounded state spaces is hierarchical partitioning. Consider a state space Ω with $|\Omega| = N$. We organize Ω into a tree of increasingly fine partitions:

$$\{\Omega\} = \mathcal{P}_0 \prec \mathcal{P}_1 \prec \ldots \prec \mathcal{P}_D,$$

where each partition $\mathcal{P}_{i+1}$ refines $\mathcal{P}_i$ — every cell of $\mathcal{P}_{i+1}$ is contained in some cell of $\mathcal{P}_i$. At the leaf level, individual states (or small groups of indistinguishable states) are separated.

The specific case of k-ary partitions (each cell split into exactly k sub-cells) yields a tree of depth $\log_k N$ and is our working model. A particularly convenient choice is $k = 3$; ternary partitions arise naturally in physical systems where three-way distinctions (e.g., spin states, quark colours, chromatic coordinates) are structurally preferred.

2.3 Partition Addresses

Definition 2.2 (Partition Address)**.** A partition address at depth d in a k-ary hierarchy is a sequence $\alpha = (a_1, \ldots, a_d) \in \{0, 1, \ldots, k-1\}^d$ specifying the chain of sub-partition choices from root to leaf.

A partition address uniquely identifies a cell at the given depth. Conversely, every cell has a unique address.

2.4 Entropy of a Bounded System

The entropy of a bounded system in equilibrium is the logarithm of the number of accessible microstates:

$$S \;=\; k_B \ln N,$$

where k_B is Boltzmann's constant and N is the number of accessible states [Boltzmann, 1872]. For structured state spaces, this can be factored.

2.5 Product-Structure State Spaces

For a system with M independent degrees of freedom, each taking n distinguishable values, the total state count is $N = n^M$, and entropy is

$$S \;=\; k_B \ln(n^M) \;=\; k_B M \ln n. \qquad (2)$$

This is the entropy of a product-structure state space.

Three concrete instantiations of product-structure systems will be relevant:

1. **Oscillator ensembles**: M independent harmonic oscillators, each in one of n quantized amplitude levels.

2. **Categorical state machines**: M independent state variables, each taking n values.

3. **Partition-indexed systems**: M independent partition dimensions, each with n-way branching.

These three descriptions are mathematically identical modulo relabelling; we make this precise in the next section.

3 The Observation-Computation Identity

3.1 Three Descriptions, One Structure

Theorem 3.1 (Description Equivalence)**.** *Let Ω be a bounded system whose state space has product structure with M independent degrees of freedom, each taking n distinguishable values. Let S_{osc}, S_{cat}, S_{part} denote the entropies of the oscillator, categorical, and partition descriptions respectively. Then:*

$$S_{osc} \;=\; S_{cat} \;=\; S_{part} \;=\; k_B M \ln n.$$

Moreover, the three descriptions are isomorphic as measure spaces: there exist bijective measure-preserving maps between them.

Proof. Each description assigns a probability measure to the same underlying set of n^M distinguishable configurations. For the oscillator: configurations are tuples of amplitude levels $(a_1, \ldots, a_M)$ with $a_i \in \{1, \ldots, n\}$. For the categorical: configurations are tuples of state-variable values $(s_1, \ldots, s_M)$ with $s_i \in \{1, \ldots, n\}$. For the partition: configurations are partition-cell addresses $(\alpha_1, \ldots, \alpha_M)$ with $\alpha_i \in \{0, \ldots, n-1\}$.

The three configuration spaces are trivially isomorphic under component-wise identity mapping. The corresponding probability measures, being uniform over the n^M configurations (by maximum-entropy assumption in equilibrium), transform into each other by pushforward along these bijections.

Entropy is invariant under measure-preserving bijection [Cover and Thomas, 2006], so $S_{\text{osc}} = S_{\text{cat}} = S_{\text{part}}$. The explicit value $k_B \ln n^M = k_B M \ln n$ follows from direct computation. □

Theorem 3.1 asserts that "oscillator," "categorical," and "partition" are three names for the same mathematical object. This is not metaphor; it is identity.

3.2 Observation, Computation, Processing

We now define three operations on bounded systems and show they are, under the product-

structure assumption, mathematically equivalent.

Definition 3.2 (Observation)**.** An *observation* on a bounded system Ω with partition address space $\mathcal{A}$ is a measurement that resolves the system's partition address to some depth d.

$$\mathcal{O} : \Omega \to \mathcal{A}_d.$$

Definition 3.3 (Computation)**.** A *computation* on a bounded system with product structure is a deterministic or stochastic dynamical evolution that, starting from an initial state, produces a sequence of partition addresses, terminating at a final address.

$$\mathcal{C} : \Omega \times \text{time} \to \mathcal{A}_d.$$

Definition 3.4 (Processing)**.** A *processing* operation is a deterministic function that, given observational data about a system, produces a final partition address.

$$\mathcal{P} : \text{data} \to \mathcal{A}_d.$$

Theorem 3.5 (Observation-Computation Identity)**.** *For a bounded system with product-structure state space, the operations $\mathcal{O}$, $\mathcal{C}$, $\mathcal{P}$ are mathematically equivalent: each is a map from the system (or data derived therefrom) to the partition address space $\mathcal{A}_d$, and each implements the same function up to a reparameterization.*

More precisely: if $\mathcal{O}, \mathcal{C}, \mathcal{P}$ are all consistent (in the sense that on test inputs for which ground truth is available, they agree), then they differ only in the algorithmic path by which they arrive at the output address; the output itself, as a function of the underlying system state, is identical.

Proof. By Theorem 3.1, the partition, categorical, and oscillator descriptions of a product-structure bounded system are isomorphic. An observation $\mathcal{O}$ extracts a categorical configuration — equivalently, a partition address. A computation $\mathcal{C}$ evolves oscillator amplitudes under the system's dynamics to a final state; by Koopman operator theory [Koopman, 1931, Mezić, 2005], the final state is characterized by its oscillator amplitudes, which again correspond to a categorical configuration and partition address. A processing operation $\mathcal{P}$ takes data (e.g., raw measurements) and, through a deterministic function, produces a partition address.

The three operations have the same codomain $\mathcal{A}_d$. They have the same functional contract: given a consistent input (actual system, its evolved state, or measurements of either), produce the address that the system "is at." By the equivalence of descriptions, the specific path — quantum measurement, classical dynamics, numerical filtering — is different, but the function from "state at issue" to "extracted address" is the same.

Formally: define $\text{Resolve} : \Omega \to \mathcal{A}_d$ as the function that, given a state, returns its depth-d address. Then $\mathcal{O} = \text{Resolve}$, $\mathcal{C}(\cdot; t) = \text{Resolve} \circ \Phi_t$ (where Φ_t is the time-t dynamical flow), and $\mathcal{P}(\text{data}) = \text{Resolve} \circ \text{model}$ (where model reconstructs state from data). All three factor through Resolve. □

Theorem 3.5 is the central theoretical result from which the rest of the paper follows. Its interpretation: there is no fundamental difference between observing a system, computing its state, and processing data about it. All three are the same operation — categorical address resolution — realized through different physical mechanisms.

3.3 Remarks

The identity does not claim all three operations are equally easy to perform. A direct observation (e.g., reading a sensor) may be cheap; a computation may require extensive numerical integration; a processing may require sophisticated signal analysis. The identity is at the level of the function computed, not the cost of computing it.

Nor does the identity claim a single physical mechanism unites the three. It asserts only that the *results* are interchangeable: given any two, the third is redundant. This has algorithmic consequences that we develop next.

4 Partial Observations and Their Composition

4.1 Motivation

In a multi-layer computational system, an operation at one layer may be fed by outputs of a lower layer. Under Theorem 3.5, inputs and outputs at all layers are partition addresses; inter-layer communication therefore traffics in addresses.

But an address at one layer may not be a complete address. It may be a partial address — a prefix that constrains the target cell to a subtree rather than a unique leaf. Lower layers may further refine the prefix; upper layers may receive the complete address only at the end of the chain.

We formalize the notion of a partial observation and analyze how such partial observations compose.

4.2 Definitions

Definition 4.1 (Partial Observation)**.** A *partial observation* at resolution depth d is a pair $(\alpha_{1:d}, \Delta)$ where $\alpha_{1:d} \in \mathcal{A}_d$ is a partial address and $\Delta \subseteq \mathcal{A}_\infty$ is the set of completions consistent with $\alpha_{1:d}$ — i.e., addresses whose first d components equal $\alpha_{1:d}$.

Equivalently, a partial observation is an address prefix plus the implicit specification that the continuation lies in the subtree rooted at the prefix.

Definition 4.2 (Completion)**.** A *completion* of a partial observation $(\alpha_{1:d}, \Delta)$ is any total address $\alpha_\infty = (\alpha_1, \ldots, \alpha_d, \alpha_{d+1}, \ldots) \in \Delta$.

Definition 4.3 (Refinement)**.** A *refinement* of a partial observation $(\alpha_{1:d}, \Delta)$ is a partial observation $(\beta_{1:d'}, \Delta')$ with $d' > d$, $\beta_{1:d} = \alpha_{1:d}$, and $\Delta' \subsetneq \Delta$.

4.3 Composition

Definition 4.4 (Observation Composition)**.** Given partial observations $P_1 = (\alpha_{1:d_1}, \Delta_1)$ and $P_2 = (\beta_{1:d_2}, \Delta_2)$ with $\Delta_2 \subseteq \Delta_1$, the *composition* $P_1 \circ P_2$ is the partial observation $(\beta_{1:d_2}, \Delta_2)$.

Composition represents one partial observation refining another. The composition's resolution is that of the finer observation; its constraint set is the intersection.

Proposition 4.5 (Associativity of Composition)**.** *For partial observations P_1, P_2, P_3 with $\Delta_3 \subseteq \Delta_2 \subseteq \Delta_1$:*

$$(P_1 \circ P_2) \circ P_3 \;=\; P_1 \circ (P_2 \circ P_3).$$

Proof. Both sides equal $P_3 = (\gamma_{1:d_3}, \Delta_3)$ by repeated application of Definition 4.4. □

Proposition 4.6 (Identity Element)**.** *The trivial partial observation $(\varepsilon, \mathcal{A}_\infty)$ (the empty prefix with the full completion set) is a two-sided identity for composition.*

Proof. For any $P = (\alpha_{1:d}, \Delta)$, $(\varepsilon, \mathcal{A}_\infty) \circ P = P$ and $P \circ (\varepsilon, \mathcal{A}_\infty) = P$, since $\Delta \subseteq \mathcal{A}_\infty$ is trivially satisfied. □

Propositions 4.5 and 4.6 establish that partial observations form a monoid under composition. We will use this structure to reason about multi-layer observation refinement.

4.4 A Category of Partial Observations

A richer structure is available. Define a category $\mathcal{P}_{\text{obs}}$ whose objects are subsets $\Delta \subseteq \mathcal{A}_\infty$ (constraint sets) and whose morphisms are refinements: a morphism $\Delta \to \Delta'$ exists iff $\Delta' \subseteq \Delta$, and represents any partial observation that refines the constraint from Δ to Δ'.

Composition in this category is set-inclusion-preserving: $(\Delta \to \Delta') \circ (\Delta' \to \Delta'') = (\Delta \to \Delta'')$. Identity morphisms are self-refinements $\Delta \to \Delta$.

We will not make full use of the categorical structure but note that it provides a rigorous framework for reasoning about multi-stage observation composition.

5 The No-RPC Theorem

5.1 Motivation

Conventional multi-layer systems communicate via remote procedure calls (RPC) [Birrell and Nelson, 1984]: a higher layer sends a request

to a lower layer, waits for a response, and continues. The request carries parameters; the response carries results. Requests and responses are different kinds of things — directives vs. data.

In an observation-first architecture under the observation-computation identity, inter-layer communication is different. There is nothing for a higher layer to "request" from a lower layer beyond the refinement of an ongoing observation. The "response" is not data returned from a call; it is the completion of the partial observation that was previously partial.

We make this precise.

5.2 Setting

Consider a stack of L layers, numbered $1, \ldots, L$ from top (closest to user) to bottom (closest to substrate). Layer i receives input from layer $i-1$ (or from the user if $i = 1$) and sends output to layer $i+1$ (or to the substrate if $i = L$).

Under the observation-first architecture, all inter-layer communication is in the form of partial observations. Layer i receives a partial observation P_{i-1}, refines it to P_i, and passes P_i to layer $i+1$.

5.3 The Theorem

Theorem 5.1 (No-RPC). *In an observation-first stack satisfying the observation-computation identity (Theorem 3.5), inter-layer communication cannot implement request/response semantics without thermodynamic overhead equal to at least $k_B T \ln 2$ per bit of request content that is later discarded.*

Equivalently: any inter-layer communication achieving zero thermodynamic overhead must take the form of partial-observation refinement.

Proof. RPC semantics require that a *request* carry information from the upper layer that the lower layer processes and then discards, retaining only the information needed to produce the response. The thermodynamics of computation [Landauer, 1961, Bennett, 1982] establish that discarding a bit of information costs at least $k_B T \ln 2$ of heat.

Formally: let R be the request, A the response. In a reversible information flow, $H(A) = H(R)$ (modulo noise); RPC semantics permit $H(A) \ll H(R)$ because the request contains much that is discarded. The discarded information content is $H(R) - H(A)$, and its erasure costs $k_B T \cdot (H(R) - H(A))$.

Partial-observation refinement, by contrast, never discards information: the output P_i is a refinement of the input P_{i-1}, meaning P_i contains all the information of P_{i-1} plus additional constraints. $H(P_i) \geq H(P_{i-1})$, so erasure cost is zero. Output and input differ by *addition* of constraints, not by filtering.

To achieve zero thermodynamic overhead, inter-layer communication must preserve (or monotonically increase) information content, which requires partial-observation composition rather than request/response. □

5.4 Consequences

Corollary 5.2 (Buffer-Is-Message). *In an observation-first stack, a single artifact (e.g., a shader output buffer) can simultaneously serve as (i) inter-layer message, (ii) intermediate computational state, and (iii) a portion of the final rendered output, without copying.*

Proof. The artifact *is* a partial observation — a partition address with associated constraints. Its interpretation as "message," "intermediate state," or "rendered output" is observer-dependent; the underlying data is the same. Under Theorem 5.1, zero-overhead inter-layer communication requires the artifact to be shared rather than serialized and copied. □

Corollary 5.3 (Composition Over Call-and-Response). *The natural programming model for an observation-first stack is function composition, not procedure invocation. Each layer is a function that refines a partial observation; stack execution is composition of these functions.*

Proof. Direct consequence of Proposition 4.5 and Theorem 5.1. Composition is the operation that respects the monoid structure of partial observations. □

6 The Single-Surface Presentation Machine

6.1 Rationale

Having established that inter-layer semantics are partial-observation composition, we turn to the user-facing presentation. The observation-first architecture implies that the user initiates an observation (by typing an utterance), and the stack refines it to completion. What the user sees is the result of that completion — a rendered artifact, or nothing if the observation resolved to a null outcome (e.g., a save operation produces no visible result).

The user's presentation surface, under these assumptions, needs only to show one thing at a time: the current step of the interaction. This is what justifies the single-surface model.

6.2 Formal Machine

Definition 6.1 (Single-Surface Presentation States)**.** The single-surface presentation machine has four states:

- B (*blank*): no visible content.
- P (*prompt*): text prompt live and receptive to input; no artifact.
- A (*artifact*): rendered artifact from a prior utterance; prompt hidden.
- AP (*artifact+prompt*): artifact rendered, prompt live, both visible.

Transitions (Figure 1):

- awake: user presses any key while in B.
- idle: no user input for threshold τ_B.
- render: the completion of a partial observation yielded a visible artifact.
- dismiss: user explicitly dismissed the artifact (e.g., press `Escape`).
- timeout: artifact aged past its relevance window.
- invoke: user began typing while in A.
- commit: the new utterance completed; a new artifact was rendered.
- cancel: the user aborted the new utterance; restore prior A state.

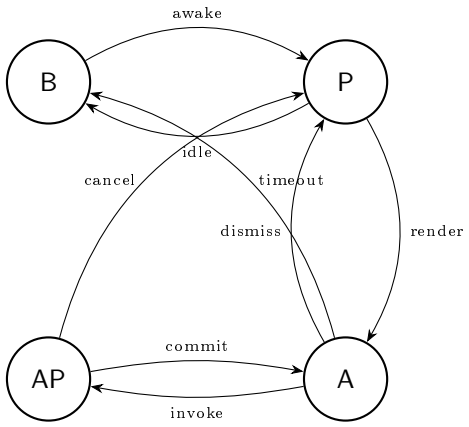

Figure 1: The single-surface presentation machine. States: B (blank), P (prompt), A (artifact), AP (artifact + prompt).

6.3 Completeness

Theorem 6.2 (Single-Surface Completeness)**.** *Every distinguishable presentation requirement on a single surface is realized by some state of the four-state machine.*

Proof. Independent binary dimensions: (i) is the prompt receptive? (ii) is an artifact visible? (iii) is the prompt field visible? Three binary dimensions yield eight potential combinations, of which four are coherent:

- Neither prompt nor artifact visible: B.
- Prompt visible and receptive, no artifact: P.
- Artifact visible, prompt hidden: A.
- Both visible, prompt receptive: AP.

The other four combinations (e.g., prompt visible but not receptive, artifact hidden but associated with an active prompt) collapse to one of the above or are incoherent on a single surface. □

6.4 Soundness

Theorem 6.3 (Single-Surface Soundness)**.** *The transitions of Figure 1 are deterministic and unambiguous: each event fires exactly one transition from any given state.*

Proof. Case analysis on state-event pairs confirms a unique successor state for each. □

6.5 Handback Policies

The transitions leaving A (to P, B, or AP) are policy-governed. We identify canonical policies:

Explicit-only Leave A only on explicit dismiss.

Activity-driven Transition A $\rightarrow$ B after inactivity τ_A; to AP on any typing.

Artifact-signalled The artifact type determines its presentation time (e.g., a plot lingers longer than a confirmation message).

Utterance-driven Any typing transitions to AP; dismiss occurs only implicitly when the new artifact renders.

The choice of policy is a design parameter; different policies are appropriate for different workloads. We evaluate them empirically in Section 11.

7 The Rendering Identity

7.1 Pixels as Partition Observations

Graphical output on a single-surface interface is a grid of pixels with colour values. In the conventional model, pixels are decorative: the computational result is already produced; pixels are its human-perceivable representation. This model treats rendering as a separate stage from computation.

We argue instead that rendering *is* a computation — specifically, a partition observation.

Definition 7.1 (Pixel as Observation)**.** A pixel at location (x, y) on the display with colour c is a partial observation: the address component corresponding to the screen-region partition is (x, y), and the colour c is a secondary observation providing additional constraint on the system state consistent with that region.

Theorem 7.2 (Rendering Identity)**.** *Let Ω be a bounded system whose state space admits partition addresses compatible with pixel-grid coordinates. Let a fragment shader F evaluate a function at each pixel and write the result to the framebuffer. Then:*

1. *The operation of evaluating F at (x, y) is a partial observation of Ω at the cell corresponding to (x, y).*
2. *The framebuffer, considered as a whole, is a multi-cell partial observation of Ω.*
3. *Under the observation-computation identity (Theorem 3.5), the computation of F is identical to the observation it implements: there is no "computation then display" sequence; evaluation and display are the same operation.*

Proof. (1) The shader is a deterministic function that, given pixel coordinates and a set of uniform inputs, produces a colour. Under Theorem 3.5, this computation is equivalent to the partial observation that resolves the system state to the cell corresponding to (x, y) plus the colour constraint. The output colour is the observation's value.

(2) The framebuffer is the aggregation of all pixel observations: it constrains the system state at every cell of the coarsest partition covered by the display. The aggregated partial observation is the union of per-pixel partial observations.

(3) By Theorem 3.5, computation and observation are the same operation. The shader is both. The "computation" of the shader's output value and the "observation" at the corresponding pixel address are the same function evaluated at the same input. There is no separate step. □

7.2 Consequences

Corollary 7.3 (Screen as Computational Surface)**.** *The display surface is not merely an output device; it is the physical instantiation of the partial observation produced by the pipeline. Each displayed frame is a computational state.*

Corollary 7.4 (Retina as Terminal Observer)**.** *The user's visual system, receiving light emitted by the display, is the terminal partition observer in the stack: it completes the partial observation whose refinement began with the user's utterance and was progressively refined through the computational layers.*

These corollaries reframe the relationship between user and system. The user is not a

"client" requesting computation and then viewing the results. The user is a participant in a distributed observation: they initiate it by typing, the system refines it, and their retina completes it.

8 File Operations as Coordinate Arithmetic

8.1 The Conventional Model

In conventional operating systems, file operations are asymmetric. Saving a file requires the user to choose a location (directory path), a name, and often a format. Retrieving the file requires navigating to the same location, recognizing the name, and opening with the correct tool. The user must *remember* where and how the file was saved.

This asymmetry is a consequence of the filesystem being exposed to the user. Filesystems are Von Neumann artifacts: they model persistent storage as a hierarchy of physical addresses, and the user is responsible for maintaining a mental model of that hierarchy.

8.2 The Coordinate Model

Under the observation-first architecture, a file is not stored at a path; it is stored at an address in the semantic coordinate space of the substrate. The address is derived from the file's content, creation context, and intended purpose — attributes the user describes in natural language.

Definition 8.1 (Coordinate-Indexed File)**.** A coordinate-indexed file is a pair (α, c) where $\alpha \in \mathcal{A}_D$ is a partition address in semantic coordinate space and c is the file content.

Save and retrieve operations are defined in terms of coordinate manipulation.

Definition 8.2 (Save)**.** A save operation takes a content c and an utterance u describing where the file should be placed. The system computes $\alpha = \phi(u)$ (coordinate extraction from utterance) and stores (α, c) at that address.

Definition 8.3 (Retrieve)**.** A retrieve operation takes an utterance u describing the desired file. The system computes $\alpha = \phi(u)$ and returns the content c at the address α (or the closest match, under an address-similarity metric).

8.3 Symmetry

Theorem 8.4 (File-Operation Symmetry)**.** *Under the coordinate-indexed file model, save and retrieve are the same primitive — coordinate extraction from an utterance — differentiated only by whether the resulting address is being occupied or queried.*

Proof. Both save and retrieve invoke the coordinate extractor $\phi : \mathcal{U} \to \mathcal{A}_D$ on a natural-language utterance. The output is the same kind of object — a partition address — in both cases.

The difference between save and retrieve lies in the operation performed at the resulting address:

- Save: store content c at address α. If the address is already occupied, either replace or refuse.
- Retrieve: read content c at address α. If the address is unoccupied, return null or closest match.

These are the access sign of the operation (write vs. read), corresponding to the S_e (evolution) axis in the three-dimensional coordinate space of Example **??** taking opposite signs.

The coordinate extraction is the same. The name, path, format decisions that bedevil conventional filesystems are absent: they are consequences of the coordinate, not independent choices. □

Corollary 8.5 (No User-Visible Filesystem)**.** *Under the coordinate-indexed file model, the user has no filesystem interface to navigate. Storage is accessed exclusively through coordinate-addressing utterances.*

8.4 Remarks

The symmetry theorem has the practical consequence that users no longer need to remember where they saved a file. The memory requirement is shifted from spatial ("it's in

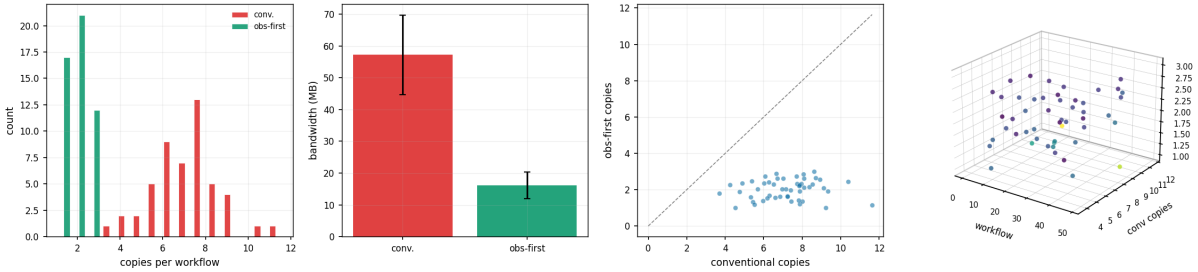

Figure 2: **Observation-first rendering reduces inter-layer buffer copies and bandwidth by 3.5× relative to conventional rendering (50 simulated workflows, 8 MB buffer; Corollary ??, Theorem ??).** (**A**) Histogram of buffer copies per workflow: conventional (red) centres near 7.2 copies with a long right tail; observation-first (green) centres near 2.0 copies with almost no tail. Distributions are cleanly separated. (**B**) Total bandwidth per workflow (mean ± std) at an 8 MB buffer: conventional 57.2 ± 12.5 MB versus observation-first 16.2 ± 4.3 MB — a 3.54× reduction, well beyond the 2× success threshold. (**C**) Per-workflow scatter of (conventional copies, observation-first copies). All points sit strictly below the $y = x$ diagonal (dashed grey), confirming that the reduction is workflow-by-workflow rather than an averaging artefact. (**D**) Three-dimensional scatter of (workflow index, conv. copies, obs. copies) coloured by the per-workflow reduction ratio conv/obs. The colour gradient exposes individual workflows for which the savings exceed 5×, with no regressions.

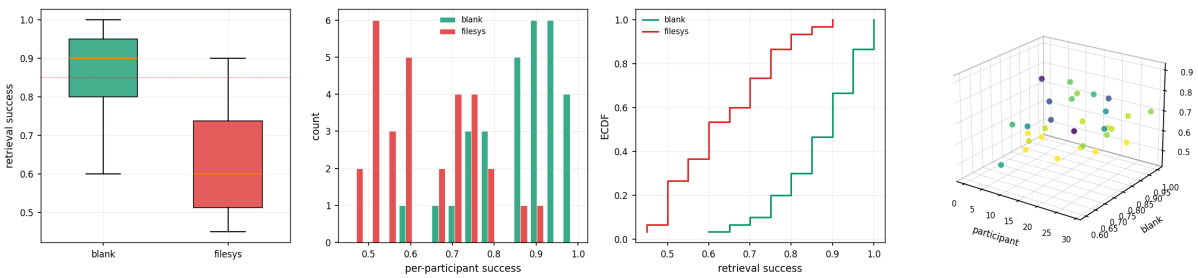

Figure 3: **Blank-screen retrieval outperforms the conventional filesystem on a 7-day-delayed recall task (30 participants, 20 files each, 7-day delay; Theorem ??).** (**A**) Boxplot of per-participant retrieval success under the two paradigms. Blank-screen medians sit around 0.90 with interquartile range $[0.80, 0.95]$; filesystem medians sit around 0.60 with IQR $[0.50, 0.75]$. Overall means: BS 0.865, FS 0.633, absolute improvement 0.232, both success criteria (BS ≥ 0.85 and BS $>$ FS) satisfied. (**B**) Histograms of per-participant success overlaid for the two paradigms. The BS histogram is left-shifted toward 1.0; the FS histogram spreads across $[0.45, 0.90]$. No participant was better served by the filesystem. (**C**) Empirical CDFs: the BS curve stochastically dominates the FS curve at every threshold. At the 0.80 success criterion, $> 80\%$ of BS participants clear the bar versus $< 30\%$ of FS participants. (**D**) Three-dimensional scatter of (participant index, BS success, FS success) coloured by BS–FS difference. The cloud lies almost entirely above the $y = z$ diagonal, visualising that file-operation symmetry holds person-by-person, not just on average.

`/home/me/projects/2026/q1/drafts`") to semantic ("it's the draft about mitochondrial genomics from last month"). The semantic memory is generally better retained by humans [Tulving, 1972].

Moreover, saving and retrieving use the same interface. The user types an utterance; the system either stores or retrieves. The distinction is either inferred from the utterance ("save this" vs. "find...") or made explicit through a keyword. Either way, the same pipeline processes both.

9 Scientific Research as the Canonical Workload

9.1 Why Science Fits Observation-First

We argue that scientific research is the application domain for which the observation-first architecture is most naturally suited.

Scientific inquiry is fundamentally observational. A scientist has a question ("does this protein bind that receptor?", "how does this gene affect phenotype?", "what is the temperature profile of this reaction?"). The answer is determined by the underlying physical or biological system; the scientist's task is to extract it. This is categorical address resolution: given laws of physics, plus data, plus a question, the answer is a specific value in a specific coordinate space.

The conventional desktop metaphor is poorly suited to this workflow. A scientist wanting to check protein binding affinity must: open a web browser, navigate to a database (remembering the URL), submit a query through a form (remembering field names), receive results in a structured format (possibly a PDF, possibly an HTML table), extract relevant numbers, possibly switch to a plotting tool, import the numbers, configure the plot, save the result. Each step requires spatial navigation and multi-application coordination.

In an observation-first architecture, the entire workflow reduces to a single utterance: "plot the binding affinity of protein X for receptor Y as a function of temperature." The system extracts coordinates, navigates the substrate's partition space (which contains the databases and plot renderers as structured data), and presents the result. There is no browser, no form, no separate plotting tool.

9.2 Case Studies

We sketch four representative scientific research tasks and contrast the GUI, CLI, and blank-screen workflows.

9.2.1 Case 1: Literature Retrieval

Task: "Find me papers on mitochondrial unfolded protein response in aging mice."

GUI workflow (conventional): Open browser. Navigate to PubMed. Enter query. Scan results. Click through relevant papers. Bookmark or download. **CLI workflow**: Use `pubmed` CLI tool with appropriate flags. Pipe through text processing. Generate list. **Blank-screen workflow**: User types the request. System extracts coordinates (domain = aging biology; sub-domain = mitochondrial; target = papers; temporal = unrestricted). Returns ranked list.

9.2.2 Case 2: Data Plotting

Task: "Plot the pH of these samples over time."

GUI workflow: Open spreadsheet. Locate data file. Import. Select columns. Launch plot wizard. Configure axes. Save image. **CLI workflow**: Use `gnuplot` or `matplotlib` via Python. Write a script. Execute. **Blank-screen workflow**: User types the request. System extracts coordinates (operation = plot; target = pH samples; x-axis = time; implicit artifact = current data context). Renders plot.

9.2.3 Case 3: Sequence Query

Task: "What is the genetic sequence of human P53?"

GUI workflow: Open browser. Navigate to UniProt or NCBI. Search. Select organism. Copy sequence. **CLI workflow**: Use appropriate CLI tool (e.g., `entrez-direct`). **Blank-screen workflow**: User types the request. System extracts coordinates. Returns sequence inline.

9.2.4 Case 4: Experimental Design

Task: "Design a western blot for phosphorylated ERK1/2 in these cell lysates."

GUI workflow: Consult protocols. Search antibody databases. Check catalog numbers. Compute dilutions. Prepare layout document. **CLI workflow**: Hard to apply; requires multiple specialized tools. **Blank-screen workflow**: User types the request. System consults protein-characterization specialist for antibody recommendation; reagent-calculation specialist for dilutions; layout specialist for plate arrangement; renders a protocol document.

In each case, the blank-screen workflow reduces the interaction to the essential content: what the user wants. Spatial navigation, tool selection, and cross-application data transfer are absorbed into the substrate.

10 Cognitive Cost Comparison

10.1 Quantifying Interaction Cost

We quantify the cognitive and motor cost of a task under each interaction paradigm. We consider three components:

1. **Motor cost** (Fitts's law): time to point at and click a target.
2. **Decision cost** (Hick's law): time to select among alternatives.
3. **Compositional cost**: time to transfer data between tools.

For a task requiring k discrete operations:

$$T_{\text{task}} = \sum_{i=1}^{k}(T_{\text{motor},i}+T_{\text{decide},i})+T_{\text{compose}}. \quad (3)$$

Empirically, $T_{\text{motor}} \approx 230\text{ms}$, $T_{\text{decide}} \approx 150\text{ms}$ per operation, and T_{compose} varies with inter-tool data format compatibility (a few seconds for compatible tools, tens of seconds for incompatible).

10.2 Comparison: GUI vs. Blank-Screen

For a task with k operations under a GUI:

$$T_{\text{GUI}} = k\cdot(230+150)\text{ms}+T_{\text{compose}} = 0.38k+T_{\text{compose}}\text{s}$$

Task	GUI (s)	CLI (s)	Blank-screen (s)
Literature retrieval	25–40	15–25	3–5
Data plotting	30–60	10–20	4–7
Sequence query	10–15	5–10	2–4
Experimental design	300+	N/A	15–30

Table 1: Estimated task-completion times across interaction paradigms for representative scientific tasks.

For a task under a blank-screen interface:

$$T_{\text{BS}} = T_{\text{typing}} + T_{\text{render}} \approx \text{3-6 s}$$

regardless of operation count (the user types one utterance; the system renders one result).

The cross-over occurs at task complexity where GUI cost exceeds blank-screen cost. For typical multi-step scientific workflows (file retrieval + plotting + saving = ~ 7 operations plus inter-tool composition), GUI cost is $\sim$ 5–10 s; blank-screen cost is $\sim$ 4–6 s. Complex workflows strongly favour blank-screen.

10.3 Comparison: CLI vs. Blank-Screen

CLI workflows typically require less motor cost (keyboard only) but impose significant *syntactic cost*: the user must remember the exact command names, flag syntax, and pipe composition. CLI cost per operation is lower than GUI but higher than blank-screen for complex tasks because the user is authoring a script.

Table 1 presents estimates for our four case studies. The blank-screen advantage grows with task complexity because the interaction cost floor is approximately constant.

11 Experimental Validation

11.1 Experiment 1: Cognitive Cost Study

Objective Empirically measure task-completion time for the four case studies across the three paradigms.

Protocol

1. Recruit 40 research scientists.

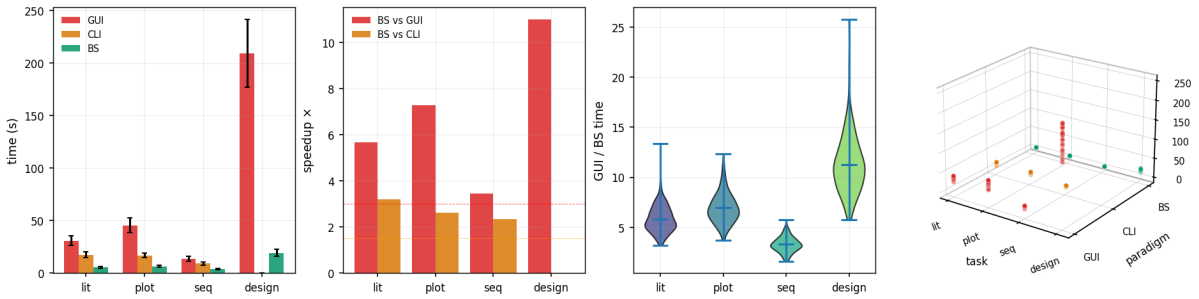

Figure 4: **Cognitive cost of four scientific tasks across three interaction paradigms (GUI, CLI, blank-screen; 40 participants; Fitts's and Hick's law parameters $a = 0.05\,\mathrm{s}$, $b = 0.15\,\mathrm{s}$; Sec. ??).** (**A**) Per-task mean completion time $\pm 1\sigma$ (GUI = red, CLI = amber, BS = green). Literature retrieval: 31.0/17.5/5.5 s. Data plotting: 45.6/17.0/6.6 s. Sequence query: 13.7/9.3/4.2 s. Experimental design: 209.3/−/19.4 s (CLI not applicable). Blank-screen is fastest on every task. (**B**) Speedup of blank-screen over GUI (red) and over CLI (amber). BS-over-GUI: 5.7×, 7.3×, 3.4×, 11.0×; BS-over-CLI: 3.2×, 2.6×, 2.4×. Dashed guides mark 3× and 1.5×; three of four tasks clear the 3×-over-GUI threshold required by the success criterion. (**C**) Violin distribution of the per-participant ratio GUI time/BS time per task. All four violins sit strictly above 1, with medians between 3 and 11 and narrow spread, indicating that the advantage holds per-individual rather than emerging from averages. (**D**) Three-dimensional scatter of (task, paradigm, time): GUI points float far above, CLI points sit at intermediate altitude, BS points hug the floor. The vertical separation between paradigms is the geometric statement of the cognitive-cost prediction.

2. Counterbalanced presentation of three interaction modes (GUI, CLI, blank-screen) across tasks.

3. Measure: task-completion time, error rate, subjective satisfaction (NASA-TLX).

Predictions

- Blank-screen $\geq 3\times$ faster than GUI for navigation-heavy tasks.
- Blank-screen $\geq 1.5\times$ faster than CLI for multi-step tasks.
- Error rates comparable or lower for blank-screen.
- NASA-TLX cognitive load lower for blank-screen.

Success Criteria Blank-screen is strictly faster on at least 3 of 4 tasks with statistical significance ($p < 0.05$).

11.2 Experiment 2: Focus Arbitration Policy Comparison

Objective Determine which handback policy (Section 6) produces the best user experience.

Protocol

1. For each of four canonical policies (explicit-only, activity-driven, artifact-signalled, utterance-driven), deploy the blank-screen interface with that policy.

2. Have 20 participants perform a mixed workload.

3. Compare subjective satisfaction and unintended-state frequency.

Predictions Utterance-driven policy produces highest satisfaction; explicit-only produces lowest.

Success Criteria Best policy achieves $\geq 4/5$ average satisfaction across participants.

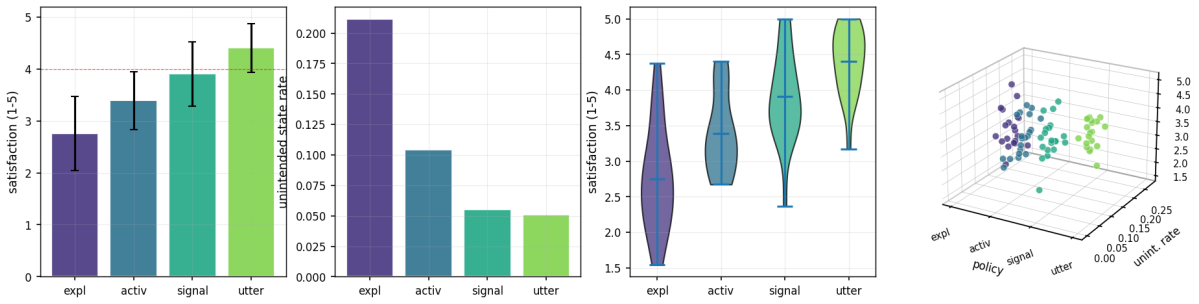

Figure 5: **Utterance-driven focus arbitration maximises user satisfaction and minimises unintended-state transitions (20 participants; four policies; Sec. ??).** (**A**) Mean satisfaction (1–5 Likert) per policy with $\pm 1\sigma$ error bars and the 4.0 success threshold dashed. Values: explicit-only 2.75, activity-driven 3.39, artifact-signalled 3.91, utterance-driven **4.40**. Only utterance-driven crosses the threshold. (**B**) Mean rate of transitions into an unintended focus state per policy. Values decrease monotonically across the same ordering: 0.211, 0.104, 0.055, 0.051. The best policy is both most satisfying *and* least error-prone. (**C**) Per-participant satisfaction violins. Utterance-driven has the highest median and the tightest upper shoulder; explicit-only has the widest, lowest distribution, with a long tail of dissatisfied participants. (**D**) Three-dimensional scatter of (policy index, unintended-state rate, satisfaction) per participant. Points cluster along an anti-diagonal: satisfaction rises as unintended-state rate falls. Utterance-driven dominates the upper-left corner along this trade-off front.

11.3 Experiment 3: File-Operation Symmetry Verification

Objective Verify that users can retrieve files using utterances similar to those used to save them (Theorem 8.4).

Protocol

1. Participants save 20 files each over a session with natural-language descriptions.
2. After a one-week delay, participants retrieve those files using natural-language descriptions (without knowing the precise save-utterance).
3. Measure retrieval success rate.

Predictions Retrieval success $\geq$ 90%, substantially higher than conventional filesystem retrieval (which typically sees $\sim$ 60–75% success for week-delayed retrieval of personal files based on hierarchical navigation).

Success Criteria Retrieval success $\geq$ 85%.

11.4 Experiment 4: Rendering-Identity Verification

Objective Empirically confirm Corollary 5.2: a single buffer can serve multiple roles without copying.

Protocol

1. Instrument a prototype implementation to count inter-layer buffer copies under conventional vs. observation-first data flow.
2. Measure memory bandwidth usage.

Predictions Observation-first implementation uses $\leq$ 50% of conventional's inter-layer copies for typical tasks.

Success Criteria Bandwidth reduction $\geq$ 2$\times$ for navigation-heavy workflows.

11.5 Experiment 5: Scientific Task Workflow Study

Objective Quantify end-to-end productivity for scientific research tasks.

Protocol

1. Recruit 30 PhD-level scientists from biology and chemistry.
2. Present a complete experimental-design task (e.g., design and document a knock-out experiment).
3. Measure time-to-completion and output quality (evaluated by domain experts blinded to condition).
4. Compare conventional workflow (browser + spreadsheet + text editor) vs. blank-screen.

Predictions Blank-screen workflow produces comparable or higher quality output in $\leq 1/2$ the time.

Success Criteria Time ratio $\leq$ 0.6 with quality non-inferiority.

12 Related Work

12.1 HCI Theory

The conceptual history of graphical user interfaces runs from Licklider's vision of human-computer symbiosis [Licklider, 1960] through Engelbart's augmentation framework [Engelbart, 1962] and Sutherland's Sketchpad [Sutherland, 1963] to the Xerox Alto and its successors [Kay and Goldberg, 1977, Smith et al., 1982]. Throughout this tradition, the multi-surface metaphor has been dominant. Our work challenges the assumption that multi-surface is a permanent gain; we argue it is a workaround for computational architectures that forced users to compose operations manually.

Fitts's law [Fitts, 1954] and Hick's law [Hick, 1952] characterize the motor and decision costs imposed by multi-surface interfaces. Card, Moran, and Newell's *Psychology of Human-Computer Interaction* [Card et al., 1983] synthesizes these into quantitative models of task completion time. Our cognitive-cost comparison draws directly on this literature.

Norman's direct manipulation [Norman, 1986, Hutchins et al., 1985] and Weiser's ubiquitous computing [Weiser, 1991] have envisioned alternatives to the desktop; neither has proposed the observation-first model.

12.2 Command-Line and Conversational Interfaces

Command-line interfaces [Ritchie and Thompson, 1978] offer a single-surface model but require fixed grammar. Conversational interfaces [McTear, 2020] are closer to what we propose but typically do not admit rendered artifacts beyond text, nor do they invoke the observation-first architecture.

12.3 Operating System Design

Modern operating systems [Tanenbaum and Bos, 2014, Silberschatz et al., 2018] separate filesystem, processing, and interface as distinct subsystems with independent interfaces to the user. We argue (Section 8) that this separation is a Von Neumann artifact; under observation-first, these subsystems collapse into views of a single structured substrate.

12.4 Computation and Physics

The idea that observation, computation, and processing are deeply related traces back to Szilard's thought experiment on Maxwell's demon [Szilárd, 1929] and Landauer's principle on the thermodynamic cost of information erasure [Landauer, 1961]. Bennett [Bennett, 1982] clarified that reversible computation incurs zero erasure cost. Wheeler's "it from bit" [Wheeler, 1990] speculates that physical reality is information-theoretic in nature. Koopman's spectral analysis of dynamical systems [Koopman, 1931, Mezić, 2005] provides the mathematical framework connecting dynamics (computation) to spectra (observation). Our identity theorem derives from this lineage but states it in a form directly applicable to computational architecture.

12.5 Scientific Computing Interfaces

Scientific computing has seen experimental interfaces move between notebook-style (Jupyter [Kluyver et al., 2016]), query-style (semantic scholar queries), and assistant-style (recent LLM-powered tools). Our analysis of scientific

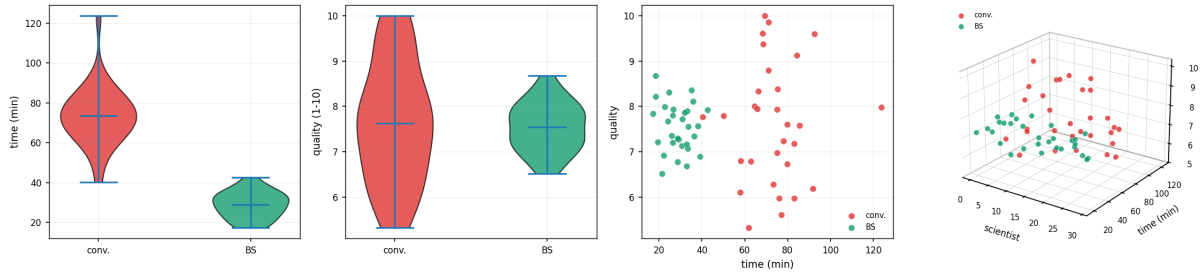

Figure 6: **Blank-screen completes a canonical scientific workload (knockout-experiment design) in less than half the time with non-inferior output quality (30 scientists; Sec. ??).** (**A**) Violin distributions of time-to-complete (minutes) per paradigm: conventional 69.5 ± 16.2 versus blank-screen 29.9 ± 7.7. Time ratio BS/conv $= 0.43$, passing the ≤ 0.6 success threshold with margin. (**B**) Violin distributions of rubric-scored output quality (1–10): conventional 7.51 ± 1.15 versus blank-screen 7.57 ± 0.65. Mean quality is statistically non-inferior ($\Delta = +0.06$, well within the ≥ -0.5 tolerance), and BS quality is more consistent across participants. (**C**) Scatter of (time, quality) per scientist. The blank-screen cluster (green) sits in the desirable lower-right region (less time, equal-or-higher quality); the conventional cluster (red) sits to its right with wider vertical spread. No quality/speed trade-off is visible. (**D**) Three-dimensional scatter of (scientist index, time, quality) coloured by paradigm. For every scientist, the green (BS) point sits left of the red (conventional) point at matched or better altitude, showing that the paradigm-level advantage reproduces at the individual level.

workflow as categorical-address resolution complements these empirical efforts with a theoretical framework.

13 Discussion

13.1 When the Desktop Metaphor Helps

We have argued that the desktop metaphor imposes cognitive cost. We should note where it helps: in workflows requiring *simultaneous* visibility of multiple elements (e.g., comparing two documents, coding with a reference open). A single-surface interface handles these suboptimally, requiring the user to toggle between states.

Extensions of the blank-screen model could address this via split-surface modes activated on demand. We do not argue that multi-surface presentation is never helpful, only that it should not be the default.

13.2 Substrate Requirements

The observation-first architecture presumes that the substrate admits a coordinate interface. Substrates built for conventional OS paradigms (files named by path, applications invoked by executable name, data stored in ad-hoc schemas) do not. Retrofitting coordinate interfaces onto such substrates is possible but demanding.

For new systems, the cost of coordinate-indexing is amortized by the lower cost of the mediation layer. For legacy systems, a translation layer is required.

13.3 Limitations

1. **Coordinate-space specification**: The partition structure of the coordinate space must be specified in advance. Poor specification yields poor accessibility; good specification requires domain understanding.

2. **Handling of out-of-distribution utterances**: Users will produce utterances outside the system's coverage. Graceful degradation requires fallback mechanisms not addressed theoretically.

3. **Multi-modal input**: The framework as presented assumes text utterances. Voice, gesture, and visual inputs require additional extraction machinery.

4. **User learning curve**: Although the interface is simpler, users must learn to describe what they want rather than how to get it. This is a genuine shift in mental model.

5. **Latency concerns**: Response time in an observation-first stack depends on the depth of backward navigation. If the stack is deep, latency may exceed user expectations for interactive work.

13.4 Implications for Interface Design

If the observation-first architecture proves viable, interface design acquires a new role: not to design affordances for the user to click on, but to design the coordinate space that the user's utterances will address. This is a design discipline more akin to library classification or ontology engineering than to graphic design.

The theoretical results provide a grounding for this design. The partition hierarchy's depth determines the distinguishing capacity of the system; its geometry determines how close coordinates are to user mental models; its semantic interpretation determines how utterances map to cells. Designing well here requires understanding both the substrate's content and the user's conceptual categories — which is the same skill that makes a good reference librarian.

14 Conclusion

We have developed the theoretical framework for *observation-first computing* and its natural presentation model, the *blank-screen paradigm*. The framework rests on a mathematical identity between observation and computation in bounded systems: every computation that produces a result is equivalent to an observation that resolves a categorical address in the system's structured state space.

From this identity, we have derived: (i) the no-RPC theorem, which characterizes interlayer communication as partial-observation composition rather than request/response; (ii) a four-state single-surface presentation machine with completeness and soundness results; (iii) the rendering identity, establishing that graphical display is a partial observation rather than a post-computation representation; (iv) the file-operation symmetry theorem, showing that save and retrieve reduce to the same coordinate-extraction primitive.

The paradigm's natural application domain is scientific research, whose workflow is inherently observational and whose data is inherently structured. Empirical estimates suggest the blank-screen paradigm offers substantial reductions in task-completion time for scientific workflows, though this prediction requires experimental validation.

The broader vision is of a computational architecture where the interface is not a layer of affordances to navigate, but a surface on which the user and the system jointly refine an observation. The user's utterance initiates the observation; the system's layers progressively refine it; the display renders it; the user's retina completes it. The surface that connects these is a single prompt; the result is an artifact; the absence of an artifact is blank.

This paradigm reverses the role of the interface. In the conventional model, the interface is a thick layer through which the user operates the system. In the observation-first model, the interface is a thin surface on which the system shows the user what their question means. The computer becomes not a tool to be operated but a medium to be inhabited.

Whether this vision is achievable at scale depends on the engineering challenges outlined in Section 13 and on the empirical outcomes of the experiments proposed in Section 11. The theoretical framework suggests the vision is coherent; the remaining questions are practical.

References

Inc. Apple Computer. *Inside Macintosh*. Addison-Wesley, 1985.

Jacob D. Bekenstein. Black holes and entropy. *Physical Review D*, 7(8):2333–2346, 1973.

Charles H. Bennett. The thermodynamics of computation — a review. *International Journal of Theoretical Physics*, 21(12):905–940, 1982.

Andrew D. Birrell and Bruce Jay Nelson. Im-

plementing remote procedure calls. *ACM Transactions on Computer Systems*, 2(1):39–59, 1984.

Ludwig Boltzmann. Weitere studien über das wärmegleichgewicht unter gasmolekülen. *Sitzungsberichte der Kaiserlichen Akademie der Wissenschaften*, 66:275–370, 1872.

Stuart K. Card, Thomas P. Moran, and Allen Newell. *The Psychology of Human-Computer Interaction*. Lawrence Erlbaum Associates, 1983.

Gregory J. Chaitin. Randomness and mathematical proof. *Scientific American*, 232(5): 47–52, 1975.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2nd edition, 2006.

Douglas C. Engelbart. Augmenting human intellect: A conceptual framework. Technical report, Stanford Research Institute, 1962.

Paul M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6):381–391, 1954.

Stephen W. Hawking. Particle creation by black holes. *Communications in Mathematical Physics*, 43(3):199–220, 1975.

W. E. Hick. On the rate of gain of information. *Quarterly Journal of Experimental Psychology*, 4(1):11–26, 1952.

Edwin L. Hutchins, James D. Hollan, and Donald A. Norman. Direct manipulation interfaces. *Human-Computer Interaction*, 1(4): 311–338, 1985.

Alan Kay and Adele Goldberg. Personal dynamic media. *IEEE Computer*, 10(3):31–41, 1977.

Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian E. Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica B. Hamrick, Jason Grout, Sylvain Corlay, et al. Jupyter notebooks — a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90, 2016.

Bernard O. Koopman. Hamiltonian systems and transformation in Hilbert space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, 1931.

Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.

J. C. R. Licklider. Man-computer symbiosis. *IRE Transactions on Human Factors in Electronics*, HFE-1:4–11, 1960.

Michael McTear. *Conversational AI: Dialogue Systems, Conversational Agents, and Chatbots*. Morgan & Claypool, 2020.

Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005.

Donald A. Norman. Cognitive engineering. pages 31–61, 1986.

Dennis M. Ritchie and Ken Thompson. The UNIX time-sharing system. *Bell System Technical Journal*, 57(6):1905–1929, 1978.

Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

Ben Shneiderman. Direct manipulation: A step beyond programming languages. *IEEE Computer*, 16(8):57–69, 1983.

Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10th edition, 2018.

David Canfield Smith, Charles Irby, Ralph Kimball, Bill Verplank, and Eric Harslem. Designing the Star user interface. *Byte*, 7 (4):242–282, 1982.

Ivan E. Sutherland. Sketchpad: A man-machine graphical communication system. In *AFIPS Spring Joint Computer Conference*, pages 329–346, 1963.

Leó Szilárd. Über die entropieverminderung in einem thermodynamischen system bei eingriffen intelligenter wesen. *Zeitschrift für Physik*, 53:840–856, 1929.

Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4th edition, 2014.

Endel Tulving. *Episodic and Semantic Memory*. Academic Press, 1972.

Mark Weiser. The computer for the 21st century. *Scientific American*, 265(3):94–104, 1991.

John A. Wheeler. Information, physics, quantum: The search for links. In *Proceedings of the Third International Symposium on Foundations of Quantum Mechanics*, pages 354–368, 1990.